Kill It and Prove It

24-Hour Engineering Hackathon: Problem Statement and Team Guide

1. The challenge in one line

Build a key-value database from scratch that survives being killed at any moment, and prove it to a judge in 15 minutes.

This is not an app-building hackathon. There is no UI, no API server, no AI feature. The entire problem is correctness under failure. A database that works when nothing goes wrong is worth zero points. A database that is still correct after being killed 500 times is what wins.

Format

Item	Detail
Duration	24 hours, continuous
Team size	2 to 4 members
Language	Any compiled or interpreted language of your choice (see Rules)
Provided by organisers	This document only. No starter code, no datasets, no servers, no cloud credits
Hardware	Your own laptops. Judging happens on your laptop
Internet	Allowed for reading documentation and using AI tools. Not allowed for downloading database libraries (see Rules)
Judging	One 15-minute live session per team at hour 24, described in Section 11

What you are being judged on

We are not measuring how fast you can produce code. We are measuring whether you understand what your code does when the process dies between two instructions. Every part of this problem is designed so that the answer must come from your own head: your test tool, your failure analysis, and your live explanation of your own code.

2. What to build

A single-process, embedded key-value database. It runs as one program, stores everything in one file on disk, and exposes the operations below. Think of it as your own tiny version of the storage layer inside SQLite or RocksDB, written by you.

Operations (all mandatory)

Operation	Behaviour	Must be durable?
`put(key, value)`	Store the value under the key. Overwrite if it exists	Yes. Once put returns success, the value survives a crash
`get(key)`	Return the current value, or "not found"	n/a
`delete(key)`	Remove the key. Deleting a missing key is not an error	Yes. Once delete returns success, the key stays gone after a crash
`batch([ops])`	Apply a list of put and delete operations atomically. Either every operation in the batch is applied, or none of them	Yes. A batch is either fully on disk or fully absent after a crash
`open(path)`	Open or create the database file and recover from any previous crash	Must complete in under 2 seconds with 1,000,000 keys
`close()`	Flush and close cleanly	Yes

Data limits

- Keys: byte strings, 1 to 1,024 bytes
- Values: byte strings, 0 to 1 MB
- Total keys: must handle at least 1,000,000 keys in one database
- Batch size: up to 10,000 operations in one batch

Concurrency

- At least two threads (or processes, your choice) must be able to call `put`, `delete` and `batch` at the same time without corrupting the file or violating atomicity.
- `get` may run concurrently with writes. A `get` must return either the value before a batch or after it, never a half-applied batch.
- You may use a single global lock. Fancy concurrency earns no extra points. Correctness does.

Storage

- Exactly one data file on disk. A separate lock file or a temporary file used during compaction is allowed, but the authoritative state must live in one file.
- You decide the on-disk format. Write-ahead log, append-only log with index, copy-on-write pages, anything you can defend.
- The file must not grow forever. After 1,000,000 puts followed by 1,000,000 deletes, the file must be able to shrink (compaction). Compaction itself must be crash-safe.

Disk full

- Your database must accept a configurable maximum file size (for example `open(path, max_bytes=50_000_000)`).
- When the limit is reached, a write must fail cleanly with a clear error, and the database must remain fully readable and recoverable. Running out of space must never corrupt existing data.
- This is how we simulate a full disk without needing special hardware.

3. Rules and constraints

Allowed

- Any programming language. Python, Go, Rust, Java, C, C++, C#, Node.js, anything.
- The language's standard library in full, including file I/O, threading, hashing, serialisation and `fsync` / `FlushFileBuffers` equivalents.
- A test framework for your own tests (pytest, JUnit, Go test, etc).
- AI tools (Claude, ChatGPT, Copilot, Cursor, anything). See Section 9 for the logging requirement.
- Reading documentation, papers, blog posts about how real databases work. We encourage it.

Not allowed

- Any database, storage engine or persistence library: SQLite, LevelDB, RocksDB, LMDB, BerkeleyDB, Redis, H2, Derby, BoltDB, Badger, and anything similar. If it stores keys and values for you, it is banned.
- Any ORM or key-value wrapper library.
- Any library that implements write-ahead logging, journaling, B-trees or LSM trees for you.
- Copying source code from an existing database project. Reading it is fine. Pasting it is not.
- Pre-written code from before the hackathon starts. Everything must be committed to a fresh repository after the start time.

Grey area, ask a judge

- Serialisation libraries such as protobuf or msgpack. Generally allowed, since they do not do storage.
- Memory-mapped file helpers. Allowed if they are part of the standard library.

Operational rules

- Everything must run offline on your own laptop. If the judge disconnects Wi-Fi and your system stops working, you score zero.
- Keep a Git repository with meaningful commits throughout. Judges will look at the commit history. One giant commit at hour 23 is a red flag.
- The judge will run your code on your laptop, from your repository, using your README. If the README is wrong, the judge stops.

Team conduct

- Teams may not share code with other teams.
- Discussion of ideas between teams is fine.

4. The invariant: the one rule that decides everything

After a crash at any point in time, and after `open()` has completed recovery:

1. Every operation that returned success before the crash is present in the database.
2. Every operation that had not returned success at the moment of the crash is either fully present or fully absent.
3. Every batch is treated as one operation for the purposes of rules 1 and 2. A batch is never partially present.
4. No key ever holds a value that was never written to it (no garbage, no torn values, no values from a different key).
5. `open()` never fails on a file that was produced by your own database, however it was killed.

That is the whole correctness requirement. Read it three times.

What "returned success" means

When your `put`, `delete` or `batch` function returns to the caller without an error, it has made a promise. The data is now safe. If the process is killed one microsecond after that return, the data must be there after restart. If your implementation returns success before the data has actually reached the disk, you have broken the promise.

What "crash" means for this hackathon

- The process is killed with no warning and no chance to run cleanup code. On Linux and macOS this is `kill -9`. On Windows it is `TerminateProcess` or Task Manager End Task.
- The kill can happen at any instruction, including in the middle of a `write()` system call, in the middle of `fsync()`, in the middle of renaming a file, or in the middle of your recovery code during the next `open()`.
- The operating system and the disk are assumed to be honest: once `fsync` returns, bytes are on disk. We are not testing for lying hardware.
- A crash during recovery must also be safe. Recovery must be able to recover from a crashed recovery.

What is NOT required

- Operations that had not returned success are allowed to be lost. If a batch was in flight and never acknowledged, it may vanish. That is correct behaviour.
- You do not need to survive the operating system crashing or power loss to the laptop. The process dying is enough. (If your design survives power loss too, say so in your document. It will impress the judges.)

5. Scenarios your database must survive

Every scenario below will be tested, either by your own torture tool or by the judge during the demo. For each one, the expected outcome after restart is stated. If your database does anything else, it is a failure.

Group A: Crash during a single write

#	Scenario	Expected after restart
A1	Kill before any bytes of a `put` reach the file	Key absent (or holds its previous value)
A2	Kill after some bytes of the record are written but not all (torn write)	Key absent or previous value. The torn bytes are ignored, never returned as a value
A3	Kill after the record is fully written but before `fsync`	Either outcome is acceptable, because success was not returned. Must not corrupt anything else
A4	Kill after `fsync` returns but before `put` returns success to the caller	Key present with the new value. The data was durable, even though the caller never heard back

#	Scenario	Expected after restart
A5	Kill immediately after `put` returns success	Key present with the new value. No exceptions
A6	Kill during a `delete`	Key either still present with old value, or absent. Never a corrupted value
A7	Kill during a `put` of a 1 MB value	Same rules as A1 to A5, with a large value spanning many disk blocks

Group B: Crash during a batch

#	Scenario	Expected after restart
B1	Kill after 3 of 10 operations in a batch are written	None of the 10 are visible
B2	Kill after all 10 are written but the commit marker is not	None of the 10 are visible
B3	Kill after the commit marker is written but before `fsync`	Either all or none. Never some
B4	Kill after the batch returns success	All 10 visible
B5	A batch that puts key X and deletes key X in the same batch	Define the ordering in your README and honour it after a crash
B6	A batch of 10,000 operations, killed halfway	None visible. Recovery still completes in under 2 seconds

Group C: Crash during recovery and maintenance

#	Scenario	Expected after restart
C1	Kill during `open()` while it is replaying the log	The next `open()` succeeds and reaches the same correct state
C2	Kill during `open()` while it is truncating or repairing a torn tail	The next `open()` succeeds
C3	Kill during compaction, before the new file replaces the old	Old file is still authoritative, all data intact
C4	Kill during compaction, in the middle of the rename or swap	Exactly one of old or new is authoritative, both are complete, nothing is lost

#	Scenario	Expected after restart
C5	Kill during compaction, after swap but before cleanup of temporary files	Data intact. Leftover temp files are cleaned up or ignored on the next `open()`
C6	Kill during `close()`	Same as a kill at any other time. `close()` is not special

Group D: Concurrency

#	Scenario	Expected after restart
D1	Two writers putting different keys, killed at random	Every acknowledged put from either writer is present
D2	Two writers putting the same key, killed at random	The key holds a value that one of the writers actually wrote. Never a mix of two values
D3	One writer in a long batch, another doing single puts, killed at random	Batch is all-or-nothing. Single puts that were acknowledged are present
D4	A reader calling `get` during a batch on the same keys (no crash)	Reader sees the state before the batch or after it. Never in between

Group E: Disk full

#	Scenario	Expected after restart
E1	File size cap reached during a single `put`	`put` returns an error. Database still readable. Previous data intact
E2	Cap reached halfway through a batch	Batch returns an error. None of its operations are visible
E3	Cap reached, then process killed	Restart succeeds. All previously acknowledged data intact
E4	Cap reached during compaction	Compaction fails or completes. Either way, no data lost and database still opens
E5	After deletes free up space, compaction runs, writes succeed again	Database is usable again without manual intervention

Group F: Repeated abuse

#	Scenario	Expected after restart
F1	500 consecutive random kills and restarts	Invariant holds after every single one
F2	Kill, restart, kill during recovery, restart, kill during recovery again	Invariant holds. Recovery is idempotent
F3	1,000,000 keys written, process killed, `open()` called	Recovery completes in under 2 seconds, all acknowledged keys present
F4	Database file truncated by 1 byte at the end by hand (simulates a lost tail)	`open()` succeeds. At most the last unacknowledged record is lost

Group G: Things that are NOT tested, so do not waste time on them

- Random bytes corrupted in the middle of the file (bit rot). Not required. Mention in your document how you would detect it if you have time.
- Network access of any kind.
- Multiple database files or sharding.
- Transactions spanning multiple batches, isolation levels, MVCC.
- Performance beyond the 2-second recovery limit. Slow writes are fine. Lost writes are not.

6. Deliverable 1: The engine

The database itself, as a library plus a small command-line wrapper so the judge can drive it without writing code.

Library interface

Expose the six operations from Section 2 in whatever idiom fits your language. Names may differ, behaviour may not.

Command-line wrapper (mandatory)

The judge must be able to run these exact commands from your repository root. Replace `./kvdb` with whatever your README says.

```
./kvdb open  <path> [--max-bytes N]      # opens or creates, runs recovery,
prints key count and recovery time
./kvdb put   <path> <key> <value>
./kvdb get   <path> <key>                 # prints value, or NOTFOUND
./kvdb del   <path> <key>
./kvdb batch <path> <file>                # file has one op per line: "put k
v" or "del k"
./kvdb dump  <path>                       # prints every key and value,
sorted by key
./kvdb check <path>                       # verifies file integrity, prints
OK or a reason
./kvdb serve <path>                       # long-running mode: reads ops from
stdin, prints ACK after each success
```

`serve` is what your torture tool and the judge will kill. It must print `ACK <op-id>` to stdout only after the operation is durable. The judge treats every `ACK` as a promise.

Repository requirements

- `README.md` with: how to build, how to run each command, the on-disk format in one page with a diagram, and your recovery algorithm in plain English.
- Build must succeed with one documented command on a clean machine with only the language toolchain installed.
- The on-disk format description must be accurate enough that a judge could write a parser from it. Judges will spot-check this against a hex dump of your file.

7. Deliverable 2: The torture tool

A program you write that tries to break your own database. This is half the problem and half the thinking. A team with a weak engine and a brilliant torture tool will score higher than a team with a decent engine and no torture tool, because the second team cannot know whether their engine works.

What it must do

1. Start your database in `serve` mode as a child process.
2. Send a stream of random operations: puts, deletes and batches with random keys and values, from at least two concurrent writers.
3. Record every operation that received an `ACK`, in a separate log file owned by the torture tool (this is the "oracle": the truth about what was promised).
4. Kill the child process with `kill -9` (or the OS equivalent) at a random moment. The moment must be genuinely random: use a random delay in milliseconds, not "after N operations".

5. Restart the database with `open`, run `check`, then `dump`.
6. Compare the dump against the oracle: every ACKed operation must be reflected, every batch must be all-or-nothing, no unknown values may appear.
7. Repeat steps 1 to 6 in a loop. Default: 500 iterations. Print a one-line summary per iteration and a final pass/fail count.
8. On the first failure, stop and print exactly what was expected versus what was found, with the operation IDs involved.

Minimum coverage

Your torture tool must have modes or flags that make it specifically target:

- Kills during large values (1 MB)
- Kills during large batches (10,000 ops)
- Kills during recovery (kill the process while `open` is running, then open again)
- Kills during compaction (trigger compaction, then kill)
- Disk-full behaviour (run with a small `--max-bytes` and verify errors are clean)

How it is run by the judge

```
./torture --iterations 500 --seed 12345
./torture --iterations 100 --seed 999 --mode large-values
./torture --iterations 100 --seed 7 --mode compaction
./torture --iterations 100 --seed 42 --mode recovery-crash
```

The `--seed` flag is mandatory. The judge will run your tool with a seed you have never used. If your tool cannot reproduce a failure from a seed, it is worth less.

What a weak torture tool looks like (and loses points for)

- Kills only between operations, never during one
- Uses `SIGTERM` or a graceful shutdown instead of `SIGKILL`
- Does not keep an independent oracle; trusts the database's own dump
- Checks only that `open` succeeds, not that the data is correct
- Never runs concurrent writers
- Never tests batches

8. Deliverable 3: The "How I would break this" document

A document of at most two pages (`BREAKING.md`) where you attack your own design in writing. This is where judges see whether you understand your system or merely produced it.

Required structure

1. **Crash points table.** List every place in your write path and recovery path where a crash is interesting. For each: the exact line or function, what is on disk if the crash happens there, and what `open()` does about it. A good submission has 8 to 15 rows. A submission with 3 rows has not thought hard enough.
2. **The ordering argument.** Explain in plain English why your sequence of `write`, `fsync` and `rename` calls guarantees the invariant. Name the specific system calls. If you fsync the directory after a rename, say so and say why. If you do not, say why it is safe anyway.
3. **Known weaknesses.** At least three honest weaknesses of your design. Things like "recovery is O(log size), so a very long log before compaction will exceed 2 seconds" or "we rely on rename being atomic, which is true on ext4 and APFS but we have not verified NTFS". Judges reward honesty here and punish claims of perfection.
4. **What we tried that failed.** The designs or bugs you abandoned, and what the torture tool found. Empty section means you either did not test or are hiding something.
5. **If we had 24 more hours.** Three concrete things you would do, in priority order.

Scoring note

Every crash point you list will be checked against your code. A crash point that is described incorrectly (you say X is on disk, the judge reads the code and sees Y) costs more than a crash point you left out. Be precise or be silent.

9. Deliverable 4: The AI prompt log

AI tools are fully allowed. We want to see how you used them.

Requirement

A file `PROMPTS.md` (or a folder `prompts/`) containing every prompt you sent to any AI tool during the hackathon, in chronological order, unedited. Include the tool name and approximate time. You do not need to include the AI's responses, only what you asked.

Why

The prompt log tells the judges a clear story. "Write me a crash-safe key-value store in Go" followed by nothing else tells us you did not engage with the problem. "My torture tool found that after a kill during compaction the old file is gone and the new one is 0 bytes. Here is my swap code. What ordering am I getting wrong?" tells us you are an engineer.

What earns credit

- Prompts that show you found a bug yourself and asked for help understanding it

- Prompts that ask the AI to explain something, followed by evidence in your code that you verified it
- Prompts where you push back on or correct the AI's suggestion

What loses credit

- Pasting this entire problem statement and submitting whatever came back
- A log that is obviously incomplete (the code has features no prompt mentions and no commit explains)
- No log at all. This is treated as "we used AI and hid it" and costs 10 points from the final score.

Honesty clause

If you used no AI at all, write that in `PROMPTS.md`. Judges may ask you to walk through any piece of code live. Code you cannot explain is scored as if an AI wrote it and you did not read it.

9A. Deliverable 5: Architecture diagram and robustness proof

A file `ARCHITECTURE.md` that proves, scenario by scenario, that your design handles every case in Section 5. `BREAKING.md` is where you attack your design. `ARCHITECTURE.md` is where you defend it.

Part 1: Architecture diagram (mandatory)

One diagram, drawn by you (hand-drawn and photographed is fine, so is draw.io, Excalidraw or Mermaid). It must show:

- Every component: client API, write path, in-memory index, log or page writer, commit marker logic, recovery routine, compaction routine, disk-full guard
- The on-disk file layout: header, record format with field sizes in bytes, commit markers, checksums, where the index lives if it is persisted
- The exact write ordering as a numbered sequence on the diagram: for example 1 append record, 2 fsync, 3 write commit marker, 4 fsync, 5 return ACK
- The recovery path as a flow: open file, validate header, scan or load index, handle torn tail, handle half-done compaction, ready
- The lock or locks that protect concurrent writers

A diagram that shows boxes called "Database" and "File" with one arrow between them scores zero.

Part 2: Scenario-to-mechanism proof table (mandatory)

A table with one row for every scenario in Section 5, Groups A to F (all 32 rows). For each row:

Column	What goes in it
Scenario	The ID, for example B2
Disk state at the moment of the kill	What bytes are and are not on disk, by reference to your format
Mechanism that handles it	The specific design element, for example "commit marker absent, recovery truncates at last valid marker"
Where in the code	File and function name
Verified by	The torture tool mode and seed that exercises it, or "manual test" with the steps, or "not verified"

"Not verified" is an honest answer and is allowed. A row that claims a mechanism the code does not actually have is not, and costs points.

Part 3: The three guarantees, argued

Three short paragraphs, one each, explaining in plain English why your architecture guarantees:

1. Durability: once ACK is sent, the data survives
2. Atomicity: a batch is never half visible
3. Recoverability: `open()` always succeeds on a file your database produced, including after a crash during recovery or compaction

Each argument must name the system calls and the ordering it depends on. "We use a write-ahead log so it is safe" is not an argument. "The commit marker is written after the record's fsync returns, so a marker on disk implies the record is fully on disk" is an argument.

How it is used by the judges

- The judge will pick three random rows from your proof table and check them against the code during the demo. One of the crash-here questions in Section 11 will come from a row you wrote.
- The architecture diagram is open on the judge's screen during the entire demo. If the code does not match the diagram, the diagram is wrong.

10. Submission format

One Git repository per team. Submit the repository URL (or a zip of the repository including `.git`) by the deadline. Late submissions are not judged.

```
team-name/
  README.md          # build, run, on-disk format, recovery algorithm
  ARCHITECTURE.md    # Deliverable 5: diagram, proof table, three guarantees
  BREAKING.md        # Deliverable 3
  PROMPTS.md         # Deliverable 4
  diagrams/          # the architecture diagram image or source
  src/               # the engine
  torture/           # the torture tool
  tests/             # any unit tests you wrote
  .git/              # full commit history from the start of the event
```

Deadline checklist

- [ ] `README.md` has a single build command that works on a clean machine
- [ ] All 8 CLI commands from Section 6 work
- [ ] `./torture --iterations 500 --seed <any>` runs to completion
- [ ] `BREAKING.md` has all 5 sections
- [ ] `PROMPTS.md` exists, even if it says "no AI used"
- [ ] Final commit is pushed before the deadline
- [ ] Laptop is charged and Wi-Fi can be turned off for the demo
- [ ] `ARCHITECTURE.md` has the diagram, all 32 proof-table rows, and the three guarantee arguments

11. The 15-minute demo

One session per team, on your laptop, with one or two judges. Wi-Fi off. The judge drives; you answer. The schedule is fixed and the judge will keep time strictly.

Minutes	What happens	What the judge is looking for
0 to 1	You build from a clean clone using the README command	Build works first time
1 to 4	Judge runs `./torture --iterations 200 --seed <judge's choice>`	Zero failures. Judge watches the output, not you

Minutes	What happens	What the judge is looking for
4 to 6	Judge runs your torture tool in `compaction` and `recovery-crash` modes with their own seeds	Zero failures
6 to 8	Judge starts `./kvdb serve`, sends a batch by hand, and kills the process manually 5 to 10 times at moments of their choosing. Then runs `check` and `dump`	Invariant holds every time. `check` says OK
8 to 12	**Crash-here questions.** Judge opens your source in an editor, points at a line in the write or recovery path, and asks: "The process dies right here. What is on disk? What does the next `open()` do?" Three different lines	Correct, specific answers, given without running the code. "I think it's fine" scores zero. "The record header is written but the body is not, so the length prefix will not match the CRC, and recovery truncates at the last valid record" scores full
12 to 15	**Design extension.** Judge says: "Add an expiry time to every key. Keys past expiry must be invisible and must be purged during compaction. Do not write code. Tell me what changes and what breaks."	A reasoned answer covering the on-disk format, recovery, and at least one subtle problem (for example, what if the clock goes backwards, or what happens to an expired key that is overwritten in a batch)

Rules during the demo

- Any team member may answer, but the judge may direct a question to a specific person. Every member should be able to explain the recovery path.
- You may look at your own code and your `BREAKING.md`. You may not use any AI tool or the internet during the demo.
- If the build fails or the torture tool crashes (the tool itself, not the database), the judge moves on and those points are lost. Test your README on a clean clone before the deadline.
- If the judge finds an invariant violation, they will show it to you and ask you to explain the cause on the spot. A correct diagnosis of your own bug recovers some of the lost points.

12. Scoring

Total 100 points. No partial credit for "it mostly works": the invariant either holds or it does not.

Area	Points	How it is scored
Survives the judge's kill runs	30	30 if zero violations across all torture runs and manual kills. Minus 10 per violation found, to a minimum of 0. A build failure scores 0 here
Torture tool quality	15	Independent oracle (4), kills during operations not between them (4), concurrent writers and batches (4), targeted modes and seed reproducibility (3)
Architecture diagram and robustness proof	10	Diagram shows components, format, write ordering and recovery flow and matches the code (4). Proof table complete and accurate on the judge's three spot-checked rows (4). Three guarantee arguments name the calls and ordering (2). Minus 2 per spot-checked row that misdescribes the code
Crash-here questions	20	Up to 7 per question (two at 7, one at 6). Full marks for a precise description of disk state and recovery action. Half for correct but vague. Zero for wrong or "it should be fine"
BREAKING.md	15	Crash points table accuracy (7), ordering argument (4), honest weaknesses and failed attempts (4). Minus 2 per crash point described incorrectly
Design extension question	10	Correct changes to format and recovery (5), identifying at least one subtle problem unprompted (4), clarity (1)

Adjustments

- Recovery slower than 2 seconds at 1,000,000 keys: minus 5
- File does not shrink after deletes and compaction: minus 5
- Disk-full behaviour corrupts data or makes the file unopenable: minus 10
- No `PROMPTS.md`: minus 10
- Commit history shows the work was not done during the event: disqualified

Tie-break

If two teams tie, the one with the stronger torture tool wins. The engine proves nothing; the tool proves the engine.

13. Disqualification

Any of the following results in disqualification, decided by the judging panel with no appeal:

- Using a banned library or storage engine, even indirectly
- Code copied from an existing database project
- Code written before the start of the event
- Sharing code between teams
- Using AI tools or the internet during the demo session
- A torture tool that is rigged to pass (for example, kills that never land during a write, or an oracle that reads from the database instead of recording ACKs independently)
- Falsifying `PROMPTS.md` or the commit history

If you are unsure whether something is allowed, ask a judge before doing it. "We did not know" is not accepted after submission.

14. Suggested timeline

You are free to work however you want. This is the plan that teams who have done this kind of thing before would follow.

Hours	Focus	Danger if skipped
0 to 2	Read this document twice. Draw your on-disk format on paper. Decide the write ordering (write, fsync, commit marker, fsync). Agree who writes the engine and who writes the torture tool, in parallel	Teams that start coding in minute 5 rewrite everything at hour 10
2 to 6	Minimal engine: append-only log, put/get/delete/batch, naive recovery by replaying the whole log. Minimal torture tool: one writer, random kills, oracle check	You need something to break by hour 6

Hours	Focus	Danger if skipped
6 to 10	Run the torture tool. It will find bugs. Fix them. Add concurrent writers. Add large values and large batches	This is where most of the learning happens
10 to 14	Compaction, crash-safe. Index for fast recovery. Hit the 2-second recovery target at 1,000,000 keys	Compaction is where the subtle swap bugs live
14 to 17	Disk-full handling. Recovery-crash mode in the torture tool. Run 500 iterations in every mode	Disk full is easy to forget and costs 10 points
17 to 20	Write `BREAKING.md` while the design is fresh. Every crash point you write down, test with the torture tool	Writing it at hour 23 produces a vague document
20 to 22	Clean clone, follow your own README, build, run everything. Fix the README. Finalise `PROMPTS.md`	A README that fails at the demo loses 30 points in the first minute
22 to 24	Rehearse the demo. Have every team member answer three crash-here questions from a teammate. Sleep if you can	Tired people give vague answers to precise questions

15. FAQ

Can we use a B-tree or LSM tree? Yes, if you write it yourself. A simple append-only log with an in-memory hash index is enough to score full marks. Complexity earns nothing by itself.

Does `fsync` really matter? Yes. Without it, data sits in the OS page cache and the process dying is survivable anyway, which hides your bugs. Your torture tool should also be run on a real disk, not a RAM disk. The judge may ask you to prove that you call `fsync` and where.

What if our language does not expose `fsync`? Every mainstream language does. Python `os.fsync`, Go `File.Sync`, Java `FileChannel.force`, Node `fs.fsyncSync`, Rust `File::sync_all`, C `fsync`. Find it.

Can the torture tool be in a different language from the engine? Yes. It talks to the engine through `serve` mode over stdin and stdout.

Can we use Docker? Only if it is already installed on your laptop and the judge can run everything with one command. Docker is not required and adds risk at the demo.

What if we run out of time and some scenarios fail? Submit anyway. Document the failures honestly in `BREAKING.md`. A team that says "compaction crash C4 fails and here is exactly why" scores far better than a team that hides it and gets caught.

Is the 2-second recovery limit strict? Yes, on the judge's run on your laptop. Design for it from the start. Replaying 1,000,000 log records sequentially is usually fast enough if your parsing is not wasteful. Measure early.

Can we ask the judges questions during the 24 hours? Yes, about the rules and the problem statement. Judges will not review your design or your code before the demo.

What should we do first? Read Section 4 again. Then draw what your file looks like after each of the seven scenarios in Group A. If you cannot draw it, you cannot build it.